

SAFE-SCAN: What the AI does and how decisions are made

SAFE-SCAN blocks return fraud using two complementary ideas: (1) create a **dispatch-time signed reference** (Golden Value) that represents the true shipped unit, and (2) run multi-signal verification at return time (embedding + FGVC + FAIM + AI Judge) while also blocking **AI-generated customer uploads** using Gemini.

Primary fraud types

AI-image claim fraud · duplicate swap · counterfeit · item swap

Evidence sources

hash + embeddings + micro-features + fusion score + LLM explanation

Outputs

verdict (AUTHENTIC / MISMATCH) + risk + confidence + key evidence

Golden Value (signed reference)

SNN-style embedding similarity

FGVC micro-features

FAIM fusion

AI Image Detection gate

AI Judge orchestration

End-to-End Flow (What happens at each stage)

1) Dispatch / OBD (create truth)

OBD captures multiple angles of the same shipped unit. SAFE-SCAN extracts a stable reference: embedding + micro-features + hash, then cryptographically signs it. This record is the “Golden Value”.

```
OBDScanner.jsx
└─ gemini.js: extractGoldenValue(images, sku, ...)
    └─ build visual fingerprint (embedding + FGVC micro-features)
        └─ imageHash / metadata
            └─ sign + store in Firebase (goldenValues)
```

2) Customer return request (gate AI-generated uploads)

Customer-uploaded photos are run through Gemini authenticity detection. If any photo is AI-generated, the return request is blocked/flagged.

```
UserReturnPortal.jsx
└─ analyzeMultipleImagesWithLogs(images)
    └─ aiImageDetection.js: detectAIGeneratedImage(base64)
        └─ Gemini returns: isAIGenerated + confidence + indicators
```

3) Pickup / FC scan (verify the unit)

The scanned return images are compared against the Golden Value. Multiple independent signals are combined, then the AI Judge makes the final decision with an explanation.

```
Scanner.jsx
└─ gemini.js: analyzeReturnScan(returnImages, goldenValue, ...)
    └─ embedding similarity (cosine/euclidean)
        └─ FGVC micro-features
            └─ optional FAIM fusion
                └─ optional AI image authenticity check
                    └─ AI Judge: finalVerdict + risk + confidence + evidence
```

Key invariant	Golden Value is verified with signature/hash before any matching is trusted.
Why multi-signal	Fraud must evade several independent checks, not just one threshold.
Explainability	AI Judge provides “why” in plain language + key indicators.

Golden Value ("Golden Number"): generation, storage, verification

The "golden number" is a signed record representing the shipped unit's visual identity. It is not a single numeric value — it's a bundle of deterministic signals and model-derived features.

What it contains

```
{
  sku, productName, category,
  imageHash,
  visualEmbedding: [512 numbers],
  fgvcAnalysis: { microFeatures: ... },
  aiMetadata: { labels, text cues, ... },
  capturedAngles: 6,
  createdAt,
  signature: "JWT (HS256)"
}
```

Why signature + hash is essential

The signature prevents tampering: if any stored field is modified later, signature/hash verification fails, and the system rejects the reference as invalid.

Hash purpose	Fast, deterministic integrity fingerprint of the golden payload (SHA-256).
JWT purpose	Cryptographic proof that the Golden Value was generated by SAFE-SCAN and not edited.
Verification gate	Verify signature/hash before using embedding/FGVC for decisioning.

AI Image Detection (Gemini): prompt-driven authenticity classification

The goal is to detect whether an uploaded image is a real camera photo vs AI-generated/rendered/stock or heavily manipulated. This is a gated decision in the customer return upload flow.

What Gemini is asked to do

Classification target	REAL_PHOTO vs AI_GENERATED (or equivalent labels).
Evidence requirement	Return indicators used for the decision (human readable).
Confidence	Return numeric confidence so operations can threshold consistently.

Signals evaluated (forensic cues)

Camera pipeline realism

sensor noise · JPEG artifacts · DoF blur

Real photos usually show imperfect noise/compression patterns and non-uniform blur.

Physical plausibility

shadows · reflections · surface contact

AI often produces inconsistent lighting physics or unnatural contact with surfaces.

Texture micro-variation

wear · dust · small scratches · repeated patterns

Diffusion outputs tend to over-smooth or repeat textures at micro scale.

Text/logos

typography plausibility · logo edges

AI may render “almost-correct” text/logos with subtle inconsistencies.

Stock-photo signals

studio background · perfect lighting

Extremely polished studio shots are suspicious in a customer-return context.

Output shape (used by UI/logging)

```
{
  isAIGenerated: true | false | null,
  confidence: 0..100,
  aiScore: 0..100,
  indicators: [...],
  verdict: "...",
  apiError?: true,
  errorType?: "RATE_LIMITED" | "API_ERROR"
}
```

SNN-Style Embedding Similarity: unit-level visual DNA

Embeddings represent images as vectors. Matching becomes a geometry problem: if the returned unit is the same, its embedding should lie close to the Golden Value embedding (after normalization).

What it detects best	item swaps and duplicate swaps where appearance differs at a global level.
Similarity measures	cosine similarity + euclidean distance (combined).
Why normalization	keeps comparisons stable across lighting/scale variation.

FGVC Micro-Features: fine-grained counterfeit & duplicate detection

FGVC focuses on fine details: logos, text sharpness, surface integrity, and small differentiators that embeddings sometimes smooth out.

Why it matters	counterfeits often look “close enough” globally but fail micro-feature consistency.
Typical features	logo accuracy · text clarity · edge sharpness · surface defects pattern.
What it outputs	micro-feature scores and a match/mismatch signal.

FAIM Fusion: combine vision with operational signals

FAIM (Fused AI for Autonomous Inventory Monitoring) combines vision scores with other signals (e.g., weight) to reduce ambiguity. This is especially useful when images alone are borderline.

Fusion idea	weighted combination of embedding similarity, FGVC, hash similarity, and optional weight checks.
Operational payoff	better separation of “close match” vs “fraud” in challenging conditions.

AI Judge: orchestration and final verdict (explainable)

AI Judge is the decision aggregator. It takes multiple algorithm outputs, detects conflicts, assigns risk, and produces an explainable final verdict.

Inputs

```
{
  hashSimilarity,
  visualSimilarity,
  snnSimilarity,
  fgvcAnalysis,
  faimScore,
  aiImageDetection
}
```

Outputs

```
{
  finalVerdict: "AUTHENTIC" | "MISMATCH" | "REVIEW",
  riskLevel: "LOW" | "MEDIUM" | "HIGH",
  confidence: 0..1,
  keyEvidence: [...],
  conflicts: [...]
}
```

This keeps the system panel-ready: you can show not only a verdict, but the chain of evidence and conflict resolution.

Thresholds & Decision Logic (operational policy)

SAFE-SCAN uses thresholds to map model signals into business actions (approve, review, block). Exact values are configurable; below is the documented policy.

AI image gate	aiScore 0–30 = real, 30–70 = review, 70–100 = AI-generated → block/flag.
Return verification	use multi-signal agreement; AI Judge resolves conflicts into verdict + risk.
Integrity precondition	Golden Value signature/hash must verify; otherwise reject as untrusted baseline.

Failures / Rate Limits (Gemini): policy and behavior

When Gemini is rate-limited or unavailable, SAFE-SCAN does not silently fall back to weak heuristics. Instead, it returns a structured error state so the UI can ask the operator/user to retry.

Why no fallback

heuristic-only detection is hard to defend and can create unacceptable false decisions.

User guidance

rate limited → "wait and retry"; API error → "service unavailable, retry scan".

Reliability stance

AI authenticity is treated as a required upstream gate, not optional decoration.
